

1 Background

Design your own simple arithmetic logic unit (ALU). Refer to the slide 58 in Lecture 7 on Arithmetic Circuits as your reference.

2 Description

- a. Inputs: a (8 bits, unsigned), b (8 bits, unsigned), F (3 bits, used to select the functions of ALU);
- b. Output: Q (8 bits, unsigned), Cout (1 bit, carry bit for Add or borrow bit for Sub);
- c. Realize the following function of ALU:
 - i. $F=000$, $Q=a+b$;
 - ii $F=001$, $Q=a-b$;
 - iii $F=010$, $Q=a \text{ OR } b$;
 - iv. $F=011$, $Q=a \text{ AND } b$;
 - v. $F=100$, $Q=a \text{ XOR } b$;
 - vi. $F=101$, $Q=\text{NOT } a$;
 - vii. $F=110$, $Q = \text{arithmetic left shift of } a \text{ by } 1 \text{ bit}$;
 - viii. $F=111$, $Q=\text{arithmetic right shift of } a \text{ by } 1 \text{ bit}$;
- d. All the ALU operations have to be finished within 1ns;
- e. You don't have to follow the schematic of slide. As far as your functionalities are the correct, it will be fine.
- f. Watch out Cout signal. It is the carry out bit for Addition and borrow bit for Subtraction. For example, for calculation of $7-3$ ", the Cout should be 0, but for calculation of $3-7$ ", the Cout should be 1. For calculation of $250+5$ ", the Cout should be 0, for calculation of $250+6$ ", Cout should be 1. One possible way is to define 9th bit of output for your internal Adder. And use the 9th bit with some additional logic to build the final Cout signal.

3 Deliverable

Please write a report with required information from below.

3.1

Following the lectures and tutorial, create all required design files including RTL, testbench, timing constraint SDC file. Attach all files to your submission.

3.2

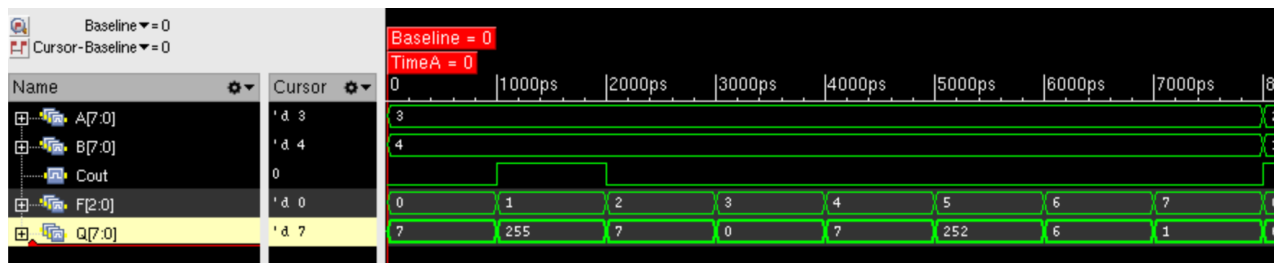
Use Xcelium to simulate your RTL using your testbench. Show the snapshots of the simulated waveforms (show all inputs and outputs signals) for each of the eight functions of the ALU. At least show one testcase (your choice) for each function of ALU. Do not use trivial cases such as all 0 inputs. Clearly mark the testing cases (mark or cycle the location of signals for the test cases) in your waveform. Correct functions should be shown.

Answer:

We simulated the RTL implementation of our 8-bit ALU in Xcelium using our custom testbench. The testbench applies non-trivial operand pairs (A,B) and sweeps the control signal F[2:0] from 000 to 111, which exercises all eight ALU operations (ADD, SUB, OR, AND, XOR, NOT, shift-left, arithmetic shift-right). Where the relation of the value of f and the operation was described in section 2. For each operation we capture both the data output Q[7:0] and the status output Cout.

Figures 1a and 1b show the waveform snapshots. In each figure, the ALU inputs A, B, and F are shown along with the outputs Q and Cout. Figure 1a uses A=3, B=4, and Figure 1b uses A=250, B=12. These are non-trivial testcases (not all zeros) as required. For each highlighted region, the observed Q and Cout match the expected ALU behavior.

The summarized numerical results extracted from these waveforms are listed in Table 1 for clearer comparison across functions.



(a) RTL simulation waveform for ALU with inputs A=3, B=4. The control field F is swept from 000 to 111.



(b) RTL simulation waveform for ALU with inputs A=250, B=12. The control field F is swept from 000 to 111.

Figure 1: RTL simulation waveform for different ALU testcases

Table 1: Gate-level ALU simulation results for two input configurations. Each Q column reports both decimal and binary.

Control (F)	Testcase 1		Testcase 2	
A	3(00000011)		250(11111010)	
B	4(00000100)		12(00001100)	
	Q	Cout	Q	Cout
000 (ADD)	7 (00000111)	0	6 (00000110)	1
001 (SUB)	255 (11111111)	1	238 (11101110)	0
010 (OR)	7 (00000111)	0	254 (11111110)	0
011 (AND)	0 (00000000)	0	8 (00001000)	0
100 (XOR)	7 (00000111)	0	246 (11110110)	0
101 (NOT)	252 (11111100)	0	5 (00000101)	0
110 (SLA)	6 (00000110)	0	244 (11110100)	0
111 (SRA)	1 (00000001)	0	253 (11111101)	0

3.3

Perform synthesis. Report your area, cell counts, and timing slack. Attach snapshot of report_area and report_timing output.

Answer:

The RTL for the 8-bit ALU was synthesized using Cadence Genus under the provided technology library and the 1 ns timing constraint (the same constraint as required by the assignment). Figure 2 summarizes the area results from `report_area`, and Fig. 3 shows the critical timing path from `report_timing`.

From `report_area`, the synthesized ALU consists of 160 standard cells. The total reported cell area is 161.728 (library units), and the estimated routed net area is 234.945, for a total physical area of approximately 396.673.

From `report_timing`, the worst-case data path (from input bit B[1] to output bit Q[7]) meets the 1 ns timing target under the `alu8_1ns.sdc` constraint. With both input and output delays set to 100 ps, the required time at the endpoint becomes 900 ps, while the actual data arrival time is 544 ps. The resulting positive slack of 356 ps, computed as

$$\begin{aligned}
 \text{Slack} &= (\text{Clock Period} - \text{Output Delay}) - (\text{Input Delay} + \text{Data Path Delay}) \\
 &= (1000 - 100) - (100 + 444) = 356 \text{ ps},
 \end{aligned}$$

indicates that the ALU comfortably satisfies the 1 ns timing requirement under typical conditions.

```

Synthesis > ≡ area.rpt
1  =====
2  Generated by:      Genus(TM) Synthesis Solution 18.14-s037_1
3  Generated on:      Oct 24 2025  01:15:24 pm
4  Module:            alu8
5  Operating conditions: typical
6  Interconnect mode: global
7  Area mode:         physical library
8  =====
9
10 Instance Module  Cell Count  Cell Area  Net Area  Total Area
11 -----
12 alu8             160         161.728    234.945    396.673

```

Figure 2: Genus report_area output for the synthesized ALU.

```

Synthesis > ≡ timing.rpt
1  =====
2  Generated by:      Genus(TM) Synthesis Solution 18.14-s037_1
3  Generated on:      Oct 24 2025  01:14:55 pm
4  Module:            alu8
5  Operating conditions: typical
6  Interconnect mode: global
7  Area mode:         physical library
8  =====
9
10 Path 1: MET {356 ps} Late External Delay Assertion at pin Q[7]
11   Group: VCLK
12   Startpoint: (F) B[1]
13   Clock: (R) VCLK
14   Endpoint: (F) Q[7]
15   Clock: (R) VCLK
16
17   Capture      Launch
18   Path Delay:++ 1000      -
19   Drv Adjust:++ 0         0
20   Src Latency:++ 0        0
21   Net Latency:++ 0 (I)    0 (I)
22   Arrival:= 1000        0
23
24   Output Delay:- 100
25   Required Time:= 900
26   Launch Clock:- 0
27   Input Delay:- 100
28   Data Path:- 444
29   Slack:= 356

```

```

Synthesis > ≡ timing.rpt
32 Exceptions/Constraints:
33 max_delay          1000      alu8_1ns.sdc_line_18
34 input_delay        100      alu8_1ns.sdc_line_10_14_1
35 output_delay       100      alu8_1ns.sdc_line_13
36
37 #-----
38 # Timing Point  Flags  Arc  Edge  Cell  Fanout Load Trans Delay Arrival Instance
39 #              (ff)  (ps) (ps) (ps) (ps) (ps) Location
40 #-----
41 B[1]           -      -    F    (arrival)  4 6.4 0 0 100 (-,-)
42 g5686__3772/ZN -      A2->ZN R  NOR2_X1  2 3.1 22 32 132 (-,-)
43 g5647__1840/ZN -      A1->ZN R  OR2_X1   1 1.2 6 26 157 (-,-)
44 g5628__2391/ZN -      A1->ZN R  AND2_X1  3 5.8 17 40 198 (-,-)
45 g5613__1857/ZN -      B1->ZN F  OAI21_X1 3 4.5 16 23 220 (-,-)
46 g5603__5266/ZN -      B1->ZN R  AOI21_X1 3 5.7 40 49 269 (-,-)
47 g5600/ZN       -      A->ZN F  INV_X1   2 2.8 12 13 283 (-,-)
48 g5595__9682/ZN -      B1->ZN R  AOI21_X1 1 2.0 23 28 311 (-,-)
49 g5592/ZN       -      A->ZN F  INV_X1   1 1.7 7 9 320 (-,-)
50 g5583__7344/ZN -      B1->ZN R  AOI21_X1 2 3.8 31 36 357 (-,-)
51 g5582/ZN       -      A->ZN F  INV_X1   2 2.8 10 13 369 (-,-)
52 g5574__5953/ZN -      B1->ZN R  AOI21_X1 2 3.8 31 37 406 (-,-)
53 g5571/ZN       -      A->ZN F  INV_X1   2 2.9 10 13 419 (-,-)
54 g5570__1309/ZN -      A1->ZN F  AND3_X1  1 1.8 7 31 450 (-,-)
55 g5565__1474/ZN -      B->ZN R  AOI211_X1 1 2.0 39 65 515 (-,-)
56 g5554__8780/ZN -      B1->ZN F  OAI221_X1 1 0.3 16 29 544 (-,-)
57 Q[7]          <<<    -    F    (port)   - - - 0 544 (-,-)
58 #-----
59

```

(a) Genus report_timing output-1.

(b) Genus report_timing output-2.

Figure 3: Genus report_timing output showing the critical path and slack.

- Cell count: 160
- Total area: 396.673 (library units)
- Worst-case path delay: 544 ps
- Required time: 900 ps
- Slack: +356 ps

3.4

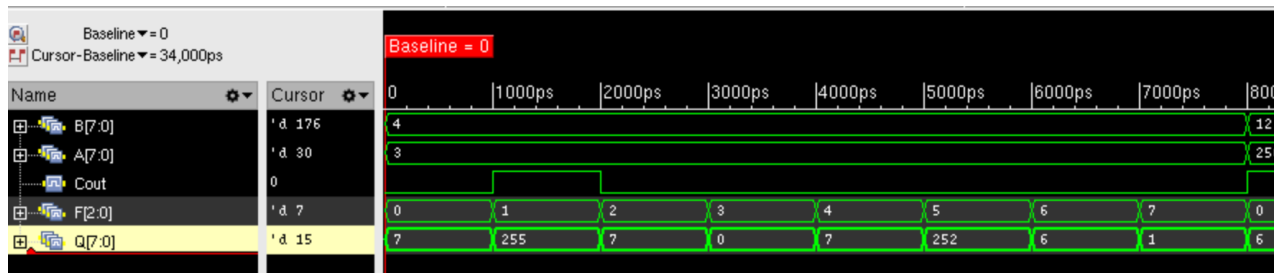
Repeat the step (2) by simulating your generated gate level netlist. Correct functions should be shown.

Answer:

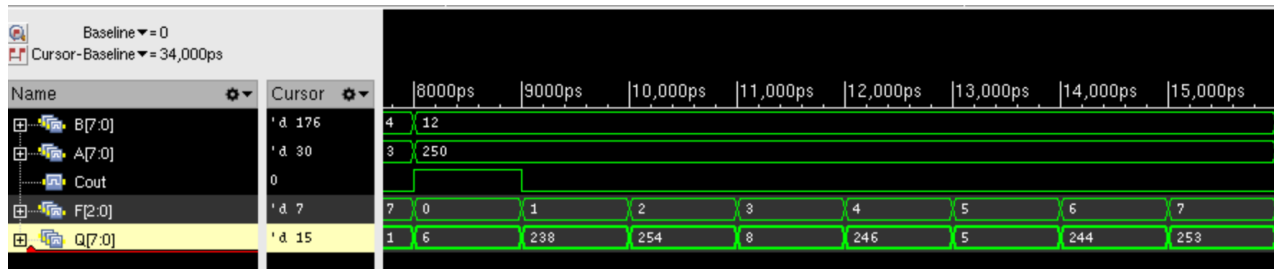
The gate-level simulation was performed using the synthesized netlist `alu_syn.v` and the same testbench `tb_alu_wave.v`. Xcelium was invoked with the following command:

```
xrun -64bit -gui -access r \
    -xmclab_args "-warnmax 0 -delay_mode zero -maxdelays" \
    ./Synthesis/alu_syn.v tb_alu_wave.v \
    /vol/ece303/genus_tutorial/NangateOpenCellLibrary.v
```

Since the simulation used the `-delay_mode zero` option, all standard-cell delays were set to zero, allowing for functional equivalence checking between the RTL and the synthesized gate-level design. The waveforms shown in Figure 4 demonstrate that the output results (`Q` and `Cout`) match those of the RTL simulation for all ALU operations (`F = 000--111`), confirming correct synthesis.



(a) Gate-level simulation waveform for ALU with inputs $A = 3$, $B = 4$. The control field F is swept from 000 to 111.



(b) Gate-level simulation waveform for ALU with inputs $A=250$, $B=12$. The control field F is swept from 000 to 111.

Figure 4: Gate-level simulation results of the synthesized ALU showing functional correctness for different testcases.

3.5

Please copy and paste into your report all supporting files your created/edited in your submission including Verilog, testbench, SDC file.

Answer:

All custom Verilog source files and constraint files used in this project are shown below, including the RTL design(Figure 5), testbench(Figure 6), and timing constraint (SDC)(Figure 7) file.

```

1  `timescale 1ns/1ps
2
3  module alu8(A, B, F, Q, Cout);
4      input [7:0] A, B;
5      input [2:0] F;
6      output [7:0] Q;
7      output Cout;
8      reg [7:0] Q;
9      reg Cout;
10     parameter ADD    = 3'b000,
11                SUB    = 3'b001,
12                ORR    = 3'b010,
13                ANDD    = 3'b011,
14                XORR    = 3'b100,
15                NOTT    = 3'b101,
16                SLA     = 3'b110,
17                SRA     = 3'b111;
18     always @ (*) begin
19         Q = 8'b0;
20         Cout = 1'b0;
21         case(F)
22             ADD:begin
23                 {Cout, Q} = {1'b0, A}+{1'b0, B};
24             end
25             SUB:begin
26                 Q = A - B;
27                 Cout = (A < B);
28             end

```

(a) ALU Verilog code (Part 1): module declaration, parameter definitions, and addition/subtraction cases.

```

3  module alu8(A, B, F, Q, Cout);
18     always @ (*) begin
21         case(F)
29             ORR:begin
30                 Q = A | B;
31             end
32             ANDD:begin
33                 Q = A & B;
34             end
35             XORR:begin
36                 Q = A ^ B;
37             end
38             NOTT:begin
39                 Q = (~A);
40             end
41             SLA:begin
42                 Q = {A[6:0], 1'b0};
43             end
44             SRA:begin
45                 Q = {A[7], A[7:1]};
46             end
47             default: begin
48                 Q = 8'd0;
49                 Cout = 1'b0;
50             end
51         endcase
52     end
53 endmodule

```

(b) ALU Verilog code (Part 2): implementation of other ALU operations.

Figure 5: Verilog RTL design of the 8-bit ALU used for synthesis

```

1  | timescale 1ns/1ps
2  module tb_alu;
3      reg [7:0] A, B;
4      reg [2:0] F;
5      wire [7:0] Q;
6      wire Cout;
7
8      alu8 dut(.A(A), .B(B), .F(F), .Q(Q), .Cout(Cout));
9
10     integer A_1 = 3;
11     integer B_1 = 4;
12     integer A_2 = 250;
13     integer B_2 = 12;
14     integer A_3 = 30;
15     integer B_3 = 176;
16
17     integer f_case;
18     integer cfg;
19     initial begin
20         for (cfg = 1; cfg <= 3; cfg = cfg + 1) begin
21             case (cfg)
22                 1: begin A = A_1; B = B_1; end
23                 2: begin A = A_2; B = B_2; end
24                 3: begin A = A_3; B = B_3; end
25             endcase
26             for (f_case = 0; f_case < 8; f_case = f_case + 1) begin
27                 F = f_case[2:0];
28                 #0.5;
29                 $display("Cfg %0d | F=%03b | A=%3d (0x%02h) | B=%3d (0x%02h) | Q=%3d (0x%02h) | Cout=%b",
30                     cfg, F, A, A, B, B, Q, Q, Cout);
31                 #0.5;
32             end
33         end
34         #10 $finish;
35     end
36 endmodule

```

Figure 6: Testbench `tb_alu_wave.v` for verifying the ALU. Sweeps all control codes ($F=000\text{--}111$) under different (A, B) configurations to validate all eight ALU operations.

```

1  | %L to chat, %K to generate
2  create_clock -name VCLK -period 1.000
3
4
5  set_input_delay -max 0.10 -clock VCLK [get_ports {A[*] B[*] F[*]}]
6  set_input_delay -min 0.02 -clock VCLK [get_ports {A[*] B[*] F[*]}]
7
8  set_output_delay -max 0.10 -clock VCLK [get_ports {Q[*] Cout}]
9  set_output_delay -min 0.02 -clock VCLK [get_ports {Q[*] Cout}]
10
11 set_max_delay 1.000 -from [get_ports {A[*] B[*] F[*]}] -to [get_ports {Q[*] Cout}]
12

```

Figure 7: Timing constraint file (`alu8_1ns.sdc`) defining a 1 ns timing requirement for the ALU. The file specifies virtual clock, input/output delays, and setup constraints.